

Ambiente Técnico: Módulo de Devoluções e Reembolsos — Plataforma OrderFlow

Author: Raja SP, AWS Labs

Source: <https://github.com/awslabs/aidlc-workflows>

Adaptation: Ricardo de Luna Galdino, EngSoft Learn

Download: [example-minimal-tech-env-brownfield.pt-br.pdf](#)

Projeto brownfield. A stack existente é a referência base. O novo código deve se encaixar

nos padrões estabelecidos. Onde uma escolha não estiver listada abaixo, siga a codebase existente — não introduza novos padrões sem justificativa.

Stack Existente (deve ser preservada)

Camada	Tecnologia Atual	Versão	Observações
Linguagem	TypeScript	5.x	Modo strict. Não introduza arquivos JavaScript.
Runtime	Node.js	20.x LTS	
Framework de API	Express	4.x	Todos os serviços existentes usam Express. Não introduza Fastify ou Koa.
Banco de dados	PostgreSQL	15	Via pg e node-postgres. Sem ORM — SQL bruto com helpers de query tipados.
Infraestrutura	AWS ECS Fargate	—	Serviços implantados como containers Docker. CDK para toda a infra.
Barramento de mensagens	Amazon SQS	—	Usado pelo notification-service para envio assíncrono de e-mails.
Autenticação	AWS Cognito	—	Tokens JWT validados no API Gateway. Não construa uma nova camada de autenticação.
Gerenciador de pacotes	npm	10.x	Não introduza yarn ou pnpm.
Framework de testes	Jest	29.x	Com ts-jest. Todos os testes em <code>__tests__</code> ao lado do código fonte.
Linter / formatador	ESLint + Prettier	—	Arquivos de configuração estão na raiz do repositório. Não os modifique.

O Que Adicionar (novo para este módulo)

- Um novo `returns-service` seguindo a mesma estrutura do `order-service`
 - Novas tabelas PostgreSQL: `return_requests`, `return_items`, `return_status_history`
 - Novos componentes React para o formulário de devolução do cliente e o painel de operações
 - Essas adições não devem modificar tabelas ou contratos de serviço existentes
-

O Que Manter Inalterado

- `order-service`, `payment-service`, `notification-service` — não modifique estes serviços
 - Tabelas PostgreSQL existentes — apenas migrações aditivas (novas tabelas, novas colunas em novas tabelas)
 - O contrato de API do `notification-service` — chame-o como documentado, não o estenda
 - Stacks CDK existentes — adicione uma nova stack para `returns-service`, não edite as stacks existentes
 - Componentes do design system do frontend — use os componentes existentes, não crie substitutos
-

O Que Remover / Não Introduzir

Proibido	Motivo	Use ao Invés Disso
ORMs (TypeORM, Prisma, Sequelize)	A codebase existente usa SQL bruto com helpers tipados. Introduzir um ORM cria inconsistência.	node-postgres com funções de query tipadas, correspondendo ao padrão existente
Axios	O projeto usa fetch nativo (embutido no Node 20).	fetch
Qualquer novo framework CSS	O frontend existente usa Tailwind CSS.	Tailwind CSS, componentes do design system existentes
Nova biblioteca de gerenciamento de estado	O frontend existente usa React Context + useReducer.	React Context + useReducer
Novo test runner (Vitest, Mocha)	O projeto usa Jest em toda parte.	Jest
Serviço ou middleware de autenticação separado	A autenticação é tratada no API Gateway via Cognito JWT.	Valide o JWT passado no cabeçalho Authorization, igual aos outros serviços

Requisitos Básicos de Segurança

- Autenticação: JWT do Cognito validado no API Gateway. Os serviços recebem os cabeçalhos `x-user-id` e `x-user-role` — confie neles, não revalide o JWT no serviço
- Autorização: Endpoints do painel de operações requerem `role === 'operations'` — verifique este cabeçalho
- Validação de entrada: Valide todos os corpos de requisição com schemas Zod antes de processar
- PII: Solicitações de devolução contêm nomes e endereços de clientes — não registre esses campos em log

- Segredos: Credenciais do banco de dados e URLs de serviços via AWS Secrets Manager, igual aos serviços existentes

Padrões de Código de Exemplo

Siga estes padrões da codebase existente. Não invente alternativas.

Um endpoint de serviço (handler de rota Express):

```
import { Router, Request, Response } from 'express';
import { z } from 'zod';
import { createReturnRequest } from '../domain/returns';
import { AppError } from '../errors';

const router = Router();

const CreateReturnSchema = z.object({
  orderId: z.string().uuid(),
  items: z.array(z.object({ orderId: z.string().uuid(), reason: z.string().min(1) })).min(1),
});

router.post('/returns', async (req: Request, res: Response) => {
  const parsed = CreateReturnSchema.safeParse(req.body);
  if (!parsed.success) {
    return res.status(400).json({ error: 'VALIDATION_ERROR', details: parsed.error.flatten() });
  }
  try {
    const result = await createReturnRequest(parsed.data, req.headers['x-user-id'] as string);
    return res.status(201).json(result);
  } catch (err) {
    if (err instanceof AppError) {
      return res.status(err.statusCode).json({ error: err.code, message: err.message });
    }
    throw err;
  }
});

export default router;
```

Uma função de query de banco de dados:

```
import { pool } from '../db/pool';

export interface ReturnRequest {
  id: string;
  orderId: string;
  customerId: string;
  status: 'submitted' | 'approved' | 'rejected' | 'refunded';
  createdAt: Date;
}

export async function getReturnRequestById(id: string): Promise<ReturnRequest | null> {
  const { rows } = await pool.query<ReturnRequest>(
    'SELECT id, order_id AS "orderId", customer_id AS "customerId", status, created_at AS "c'
    reatedAt" FROM return_requests WHERE id = $1',
    [id]
  );
  return rows[0] ?? null;
}
```

Um teste Jest:

```
import { getReturnRequestById } from '../db/return-requests';
import { pool } from '../db/pool';

jest.mock('../db/pool');
const mockQuery = pool.query as jest.Mock;

describe('getReturnRequestById', () => {
  it('returns the request when found', async () => {
    mockQuery.mockResolvedValueOnce({ rows: [{ id: 'abc', orderId: '123', status: 'submitt'
    e' }] });
    const result = await getReturnRequestById('abc');
    expect(result?.id).toBe('abc');
  });

  it('returns null when not found', async () => {
    mockQuery.mockResolvedValueOnce({ rows: [] });
    const result = await getReturnRequestById('missing');
    expect(result).toBeNull();
  });
});
```